7

Лекция 23–24 марта

Регулярные выражения

Константин Чухарев

Дискретная математика — Весна 2027

Регулярные выражения

> *Аналитическая машина ткёт алгебраические узоры так же, как жаккардовый станок ткёт цветы и листья.*
>
> ***— Ада Лавлейс***

Два описания языка

Автомат — *распознаватель*: читает слово и отвечает, принадлежит ли оно языку.

Нужна и *запись*: короткая формула, из которой язык восстанавливается без машины.

Стивен Клини ввёл регулярные выражения в 1951 году, чтобы не рисовать диаграммы: язык, заданный кружками и стрелками, хотелось сжимать до строки, которую можно продиктовать по телефону.

Автомат исполняет, выражение записывает.

Теорема Клини покажет, что это две стороны одного класса языков.

Регулярное выражение

Определение 1

Регулярное выражение над Σ строится по правилам:

- $\varnothing$ — пустой язык;
- ε — язык $\{\varepsilon\}$ из одного пустого слова;
- $a \in \Sigma$ — язык $\{a\}$;
- $R_1 \mid R_2$ — объединение языков;
- $R_1 R_2$ — конкатенация;
- R^* — звезда: ноль и более повторений.

Пример

Приоритет от сильной к слабой: $*$, конкатенация, $|$.

Скобки меняют язык: $(a \mid b)c$ порождает a c и b c, а $a \mid bc$ — a и b c.

Синтаксис: приоритет и скобки

Приоритет разрешает двусмысленность записи: каждый символ связывается с операцией однозначно.

Запись	Читается как	Язык
ab^*	$a(b^*)$	слово a, затем сколько угодно b
$ab \mid c$	$(ab) \mid c$	слово ab или слово c
$a \mid bc$	$a \mid (bc)$	слово a или слово bc

Похожие записи задают разные языки: ab^* и $(ab)^*$.

Слово abb лежит в первом и не лежит во втором: там повторяется пара ab, а не буква b.

Разбор выражения: дерево для $(a \mid b)^*abb$

Пример

Соберём дерево разбора для $(a \mid b)^*abb$:

1. звезда связывает сильнее всех: $(a \mid b)^*$ — узел с выбором $a \mid b$ внутри;
2. конкатенация собирает четыре множителя и группирует их слева: $((a \mid b)^*ab)b$.

```
concat
  concat
    concat
      star(a | b)
      a
    b
  b
```

Узлов в дереве — линейно от длины записи, поэтому автомат Томпсона выходит из $O(n)$ состояний.

Примеры выражений

Пример

Над алфавитом $\{0, 1\}$:

- $(0 \mid 1)^*$ — все двоичные слова, включая пустое: 0, 110, ε.
- 01^*0 — слова с нулями по краям и единицами в середине: 00, 010, 0110.
- $(0 \mid 1)^*01$ — слова с хвостом 01: 01, 001, 1101.
- $(0 \mid 1)(0 \mid 1)^*$ — все непустые слова: 0, 10, 011.

$\emptyset$, ε и 0^* — три разных языка: в первом нет слов, во втором слово длины 0, в третьем слова из нулей любой длины.

Язык выражения: структурная рекурсия

Определение 2

Язык $L(R)$ выражения R определяется вместе с выражением:

- $L(\emptyset) = \emptyset$, $L(\varepsilon) = \{\varepsilon\}$, $L(a) = \{a\}$ для $a \in \Sigma$;
- $L(R \mid S) = L(R) \cup L(S)$;
- $L(RS) = \{uv \mid u \in L(R), v \in L(S)\}$;
- $L(R^*) = \cup_{k \geq 0} L(R)^k$, где $L(R)^0 = \{\varepsilon\}$.

Пример

Вычислим язык выражения $(a \mid b)^*abb$ по шагам:

$$L((a \mid b)^*abb) = \{a, b\}^*\{a\}\{b\}\{b\} = \{uabb \mid u \in \{a, b\}^*\},$$

то есть все слова, заканчивающиеся на abb.

Слова aabb и abb лежат в языке, а ab и abab — нет.

Алгебра регулярных выражений

Тождество выражений — равенство двух языков, и проверяется оно как равенство множеств слов.

Тождество	Имя
$\varepsilon R = R\varepsilon = R$	нейтральность пустого слова
$\varnothing R = R\varnothing = \varnothing$	аннулирование
$\varepsilon \mid RR^* = R^*$	развёртка звезды
$(R^*)^* = R^*$	идемпотентность звезды

Доказательство — взаимное включение: слово раскладывается по построению и собирается обратно.

Тождества: наброски доказательств

Пример

Докажем ассоциативность $(RS)T = R(ST)$ и дистрибутивность $R(S \mid T) = RS \mid RT$.

Ассоциативность: слово любой из сторон равно uvw при $u \in L(R)$, $v \in L(S)$, $w \in L(T)$.

Дистрибутивность: слово слева равно uv, где v лежит в $L(S)$ или в $L(T)$.

Пример

Развёртка $\varepsilon \mid RR^* = R^*$: слева пустое слово либо $r_1(r_2 \ldots r_k)$ при $k \geq 1$ — оба лежат в R^*, а $RR^* \subseteq R^*$ по определению.

Для $R = a$: $\varepsilon \mid aa^* = a^*$.

Идемпотентность звезды

Пример

Идемпотентность звезды: $(R^*)^* = R^*$.

Слева направо: слово из $(R^*)^*$ — произведение кусков, каждый из которых лежит в R^*, а произведение слов из R^* само лежит в R^* по определению звезды.

Обратно: слово из R^* берётся в $(R^*)^*$ с одним повторением куска, и лишняя обёртка ничего не добавляет.

Проверка на $R = a \mid b$: оба языка совпадают с $(a \mid b)^*$, и слово $\mathrm{b\ a\ b}$ лежит в каждом.

Вопросы для самопроверки — Алгебра

1. Упростите $(a \mid \varepsilon)^*$ и обоснуйте упрощение тождествами таблицы.
2. Проверьте на словах тождество $(RS)^*R = R(SR)^*$.

Контрпримеры: похожие законы, которые ложны

Пример

Закон $(R \mid S)^* = R^* S^*$ похож на дистрибутивность и ложен.

Для $R = \{a\}$ и $S = \{b\}$ слово b a лежит в $\{(a \mid b)\}^*$, но не в $a^* b^*$: там все a идут раньше всех b.

Конкатенация некоммутативна: $\{a\}\{b\} = \{ab\}$, а $\{b\}\{a\} = \{ba\}$ — разные языки.

Арифметическая интуиция переносится на языки выборочно.

Теорема Клини

Класс языков автоматов обозначается AUT, класс языков выражений — REG.

Теорема 1 (Клини, 1956)

Язык регулярен тогда и только тогда, когда он представим регулярным выражением:

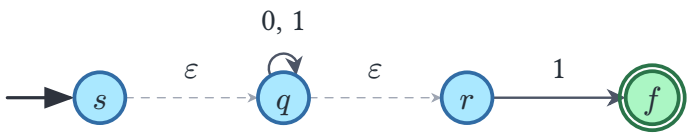
$$\mathrm{AUT} = \mathrm{REG}.$$

Доказательство

Докажем двумя построениями, по одному на направление. От выражения к автомату: структурная индукция — атомы дают кирпичи из пары состояний, операции склеивают их ε-переходами. От автомата к выражению: состояниям приписываются выражения для слов, ведущих из старта, и язык собирается рекуррентностью Клини.

От выражения к автомату

- База: атомы a, ε, $\emptyset$ — автоматы из пары состояний.
- Объединение и конкатенация — ε-переходы: параллельный запуск и последовательное соединение.
- Звезда — возврат из конца в старт плюс обход пустого слова.



Сборка и прогон $(0 \mid 1)^*1$

Пример (Полная сборка)

Автомат собран из кирпичей: из s в q ведёт ε-ребро входа, петля q читает 0 и 1, из q в r — ε-ребро выхода, из r в f — ребро по 1. Принимающее состояние одно: f.

Прогон слова 01: старт — ε -closure$(\{s\}) = \{s, q, r\}$. По 0 ведёт только петля q, и замыкание добавляет r: живо $\{q, r\}$. По 1 петля держит q, а r переходит в f, и замыкание возвращает r по ε-ребру из q: живо $\{q, f, r\}$, слово принято.

Пустое слово кончается в $\{s, q, r\} - f$ не достигнут, и звезда честно не принимает ε.

От автомата к выражению

Алгоритм Клини: состояния нумеруются, и через R_{ij}^{k} обозначаются слова, переводящие из q_i в q_j путями с промежуточными состояниями не выше k.

База — переходы автомата, шаг — рекуррентность:

$$R_{ij}^{k} = R_{ij}^{k-1} \cup R_{ik}^{k-1} \left(R_{kk}^{k-1}\right)^{*} R_{kj}^{k-1}.$$

Путь либо не заходит в q_k, либо проходит его цикл сколько угодно раз.

Это тот же приём, что в алгоритме Флойда—Уоршелла: динамическое программирование по промежуточным вершинам.

Разница в алгебре: вместо min и сложения — объединение, конкатенация и звезда.

Выражение выходит огромным, но полное доказательство существует для любого автомата.

Устранение состояний

Есть и второй путь от автомата к выражению: удалять состояния по одному, перенося их работу на рёбра.

Определение 3

Рёбра автомата помечаются регулярными выражениями, добавляются вход s с ε-переходом в старт и выход t с ε-переходами из принимающих состояний.

Устранение состояния q превращает каждое ребро $i \to j$ в обход через q:

$$R_{ij} \quad \cup \quad R_{iq}\left(R_{qq}\right)^{*} R_{qj}.$$

Когда состояний не остаётся, метка единственного ребра $s \to t$ — выражение языка.

Формула — та же рекуррентность алгоритма Клини: путь либо обходит q, либо прокручивает его петлю, а состояния устраняются целиком.

Устранение: автомат для $(a \mid b)^*ab$

Автомат для шаблона ab: три состояния и одно принимающее.

Состояние	**по a**	**по b**
q_0 — старт	q_1	q_0
q_1 — хвост a	q_1	q_2
q_2 — принято	q_1	q_0

Пример

Шаг 1: устраняем q_1. Входы в него — рёбра a из q_0 и q_2, петля a, а выход — ребро b в q_2, поэтому ребро $q_0 \to q_2$ получает метку aa^*b: войти по a и сколько угодно ещё a, выйти по b.

Петля q_2 получает ту же метку aa^*b, а ребро $q_2 \to q_0$ по b остаётся без изменений.

Прогон b a b: $q_0 \to q_0 \to q_1 \to q_2$ — принято, a a b b: $q_0 \to q_1 \to q_2 \to q_0$ — отвергнуто.

Устранение: шаги 2 и 3

Пример

Шаг 2: устраняем q_0. Входы — ε от входа s и b из q_2, выходы — петля b и ребро aa^*b.

$$s \to q_2 : \varepsilon b^*(aa^*b) = b^*aa^*b, \quad q_2 \to q_2 : aa^*b \cup bb^*(aa^*b).$$

Шаг 3: устраняем q_2, и метка единственного ребра $s \to t$ равна $b^*aa^*b(aa^*b \cup bb^*aa^*b)^*$.

Петлю упрощает развёртка звезды: $aa^*b \cup bb^*aa^*b = (\varepsilon \mid bb^*)aa^*b = b^*aa^*b$.

Выражение $b^*aa^*b(b^*aa^*b)^*$ задаёт тот же язык, что $(a \mid b)^*ab$: кусок b^*aa^*b — слово, заканчивающееся на ab, а звезда его повторяет — законное тождество алгебры выражений.

На словах: a a b b a b распадается на куски a a b и b a b, и каждый кусок имеет вид b^*aa^*b.

Замыкания: булевы операции

Что остаётся регулярным после операций над языками?

Теорема 2 (Замкнутость относительно булевых операций)

Если L_1 и L_2 регулярны, то регулярны $L_1 \cup L_2$, $L_1 \cap L_2$, $\overline{L_1}$ и $L_1 \setminus L_2$.

Доказательство

Докажем конструкциями, по одной на операцию.

Объединение — склейка двух ε-автоматов или запись $R_1 \mid R_2$.

Дополнение: путь у ДКА один, и смена принимающих состояний меняет ответ.

Пересечение — прямое произведение: пары (q, r) с покомпонентными переходами и принимающими парами, а разность — пересечение с дополнением: $L_1 \setminus L_2 = L_1 \cap \overline{L_2}$.

Дополнение: смена принимающих состояний

Пример

У ДКА для $(a \mid b)^*ab$ принимающее состояние одно — q_2, остальные два состояния не принимают. Меняем роли: принимающими становятся q_0 и q_1, и автомат распознаёт все слова, которые *не* заканчиваются на ab.

Слово a b ушло из языка, слова a b a и ε появились: путь у ДКА один, и для каждого слова ответ перевернулся.

Для НКА тот же приём ломается: путей много, и слово с одним неуспешным путём может иметь и принимающий.

Пересечение: прямое произведение

Пример

L_1 — слова, заканчивающиеся на 1: ДКА с состояниями p_0 и p_1, принимающий p_1. L_2 — слова чётной длины: ДКА с состояниями e_0 и e_1, принимающий e_0. Произведение живёт в парах (p, e), читает символ покомпонентно и принимает пары (p_1, e_0) — язык слов, которые заканчиваются на 1 и имеют чётную длину.

Слово 0 1 идёт $(p_0, e_0) \to (p_0, e_1) \to (p_1, e_0)$ и принимается, слово 1 0 1 приходит в (p_1, e_1) и отвергается.

У трёх операций выражений нет знака пересечения: замыкание видно только на автоматах.

Производные выражений

Определение 4

Производная выражения R по символу a — выражение $D_a(R)$ с языком $\{w \mid aw \in L(R)\}$.

- $D_a(\varnothing) = D_a(\varepsilon) = \varnothing$, $D_a(a) = \varepsilon$, $D_a(b) = \varnothing$ при $b \neq a$;
- $D_a(R \mid S) = D_a(R) \mid D_a(S)$ и $D_a(R^*) = D_a(R)R^*$;
- $D_a(RS) = D_a(R)S \mid \nu(R)D_a(S)$, где $\nu(R) = \varepsilon$ при $\varepsilon \in L(R)$ и $\varnothing$ иначе.

Теорема 3 (Бржозовски, 1963)

Слово w лежит в $L(R)$ тогда и только тогда, когда $\varepsilon \in L(D_w(R))$, где D_w — производные по буквам слова. Различных производных у выражения конечное число.

Значения ν видны по записи: $\nu(\varepsilon) = \nu(R^*) = \varepsilon$, $\nu(a) = \nu(\varnothing) = \varnothing$.

Производные: пример

Пример

Выражение $R = a^*b$, текст a a b.

$D_a(R) = D_a(a^*)b \mid \nu(a^*)D_a(b) = a^*b$ — первые две буквы производную не меняют.

$D_b(a^*b) = \varnothing \mid \varepsilon = \varepsilon$: язык производной содержит пустое слово, текст принят.

Пример

Текст a b a: после a b производная равна ε, а $D_a(\varepsilon) = \varnothing$ — текст отвергнут.

Различных производных у a^*b ровно три: a^*b, ε и $\varnothing$ — три состояния ДКА для того же языка, пересчитываемые прямо на записи.

Регулярные выражения вокруг нас

Пример

Идентификатор языка программирования: `[a-zA-Z_][a-zA-Z0-9_]*` — буква или подчёркивание, затем любые символы имени.

Тот же язык: буква (буква | цифра)*.

- Валидация форм: телефон, почтовый индекс, адрес почты.
- Поиск и замена в редакторах и утилитах командной строки.
- Подсветка синтаксиса и генераторы лексеров.

Всё это — теорема Клини в действии: запись компилируется в автомат, автомат исполняет распознавание.

Инженерия: от grep до лексера

Кен Томпсон скомпилировал регулярное выражение в автомат для редактора QED в 1968 году.

Тот же приём переехал в `ed`, затем в `grep`.

Лексер — первый компонент каждого компилятора: токены задаются регулярными выражениями, компилируются в ДКА и распознаются за $O(1)$ на символ.

Регулярные языки заканчиваются на лексере: вложенные скобки и грамматика языка требуют более сильных моделей.

Библиотеки с возвратом перебирают пути автомата по одному — на неудачных парах выражение–текст время взрывается экспоненциально.

В теории: выражение компилируется в ДКА, и распознавание линейно по тексту.

Движки: POSIX против возврата

Запись одна, движков два семейства, и совпадение они выбирают по-разному.

Свойство	Автоматные	С возвратом
Стратегия	симуляция НКА множеством	обход дерева в глубину
Худший случай	линейное время	экспоненциальное время
Правило выбора	наибольшее (POSIX) либо первое (Perl)	по порядку альтернатив
Обратные ссылки	нет	есть, и язык уже нерегулярен

Пример

Шаблон `a | ab` на слове `ab`: совпадения в первой позиции — `a` и `ab`.

POSIX-движки (`grep -E`, `awk`) вернут `ab`: наибольшее совпадение.

PCRE, Python `re`, RE2 и Rust `regex` по умолчанию вернут `a`: первая альтернатива.

Катастрофический возврат

Выражение $(aa^*)^*b$ задаёт цепочку из a с одной b в конце — его ДКА имеет три состояния.

Пример

Для k повторений внешней звезды выходит $\binom{n-1}{k-1}$ разбиений цепочки из n букв.

Сумма по всем k равна 2^{n-1}: тридцать букв — $2^{29} \approx 5.4 \times 10^8$ попыток.

```
import re

pattern = re.compile(r"(a+)+b")
pattern.match("a" * 30)   # 2^29 разбиений --- минуты счёта
```

Перебор идёт по одному пути, и все пути несут одно и то же: «прочитаны a, ждём b».

В июле 2019 года такой возврат в WAF-фильтре Cloudflare отключал сайты на полчаса.

Нерегулярные языки

> *В одну и ту же реку нельзя войти дважды.*
>
> ***— Гераклит***

Конечная память имеет границу

Автомат не умеет считать неограниченно.

Пример

Язык $L = \{0^n 1^n \mid n \geq 0\}$: поровну нулей и единиц, все нули раньше.

Для распознавания нужно помнить разность «нули минус единицы» — а она ничем не ограничена.

«Не регулярен» — утверждение о всех автоматах сразу: ни один ДКА не распознаёт язык.

Для такого утверждения нужен инструмент — свойство, которым обладает *каждый* регулярный язык.

Цикл в диаграмме переходов

Пусть ДКА имеет p состояний и читает слово длины p.

Пройдено $p + 1$ позиций, а состояний p — по принципу Дирихле какое-то состояние повторилось.

Пример

Автомат с тремя состояниями читает 0001.

Пять позиций пути, три состояния: где-то автомат вернулся в то же место — и дальше идёт по той же колее, сколько бы раз ни прошёл цикл.

Повтор состояния — цикл, который можно пройти ноль, один или много раз.

Автомат разницы не заметит: конец пути тот же.

Лемма о накачке

Теорема 4 (Лемма о накачке)

Если язык L регулярен, то найдётся число p, такое что любое слово $w \in L$ длины $|w| \geq p$ разбивается как $w = xyz$, где:

- $|y| > 0$;
- $|xy| \leq p$;
- $xy^i z \in L$ для всех $i \geq 0$.

Доказательство

Докажем от принципа Дирихле.

Пусть ДКА для L имеет p состояний, на первых p символах слова w какое-то состояние s повторяется. Пусть x — префикс до первого вхождения s, y — цикл между двумя вхождениями, z — остаток. Тогда $|y| > 0$, $|xy| \leq p$, и накачка $xy^i z$ водит автомат по одному циклу — исход не меняется.

Накачка работает и на регулярных

Пример

У автомата для слов, заканчивающихся на 01, всего $p = 3$ состояния, а слово 0001 ведёт его по состояниям q_0, q_1, q_1, q_1, q_2.

Состояние q_1 повторилось, и разбиение готово: $x = 0$, $y = 0$, $z = 01$. Условия $|y| > 0$ и $|xy| \leq 3$ выполнены, а накачанные слова 001, 0001, 00001 — все кончаются на 01.

Лемма — свойство *каждого* регулярного языка: в нём найдётся накачиваемое разбиение каждого длинного слова.

Значит, если для языка разбиения нет ни при каком p — язык не регулярен.

Опровержение: язык $0^n 1^n$

Пример

Пусть $L = \{0^n 1^n\}$ регулярен с длиной накачки p.

Возьмём $w = 0^p 1^p \in L$. По лемме $w = xyz$ с $|xy| \le p$, $|y| > 0$.

1. Префикс xy целиком лежит в первых p символах — состоит только из нулей.
2. Значит, $y = 0^k$ для некоторого $k > 0$.
3. Накачанное слово $xy^2z = 0^{p+k}1^p$ должно лежать в L.
4. Но нулей в нём больше, чем единиц, — противоречие.

Следовательно, L не регулярен.

Схема опровержения: взять слово подлиннее p, накачать, получить слово вне языка.

Другие нерегулярные языки

- Палиндромы над $\{a, b\}$: слова a^n разводятся продолжением $ba^n - a^i ba^i$ палиндром, $a^j ba^i$ при $j \neq i$ нет.
- a^{2^n}: накачка $y = a^k$ даёт длины между 2^p и 2^{p+1}, а степеней двойки там нет.
- $a^n b^n c^n$: накачка в первых p символах портит блок a, а остальные блоки стоят на месте.

В каждом опровержении та же анатомия: длинное слово, накачка в префиксе, выход из языка.

Меняется только арифметика разрушения.

Границы леммы

Лемма о накачке даёт необходимое условие регулярности, но не достаточное.

Существуют нерегулярные языки, которые удовлетворяют лемме: на них опровержение не выходит.

Опровергнуть регулярность — можно, доказать — нет.

Конструктивный инструмент, работающий в обе стороны и строящий минимальный автомат, — теорема Майхилла–Нерода.

Что мы поняли?

Выражение и автомат — две записи одного класса.

Замыкания держат операции внутри класса, накачка отмечает его границу.

- Регулярное выражение — запись языка из атомов и трёх операций: $|$, конкатенация, звезда.
- Теорема Клини: $\mathrm{AUT} = \mathrm{REG}$, два описания — один класс.
- Туда и обратно: конструкция Томпсона и алгоритм Клини с устранением состояний.
- Класс замкнут: дополнение — смена принимающих ДКА, пересечение — прямое произведение.
- Накачка: длинное слово регулярного языка содержит накачиваемый цикл, а $0^n 1^n$ нарушает условие леммы.

Вопросы для самопроверки

Вопросы для самопроверки — выражения, автоматы и накачка

1. Запишите выражение для слов над $\{0, 1\}$ с подстрокой 01.
2. Чем различны языки $\emptyset$ и ε и что содержит $\emptyset^*$?
3. Что в алгоритме Клини играет роль промежуточных вершин из Флойда—Уоршелла?
4. Накачкой докажите нерегулярность языка $a^n b^n c^n$.
5. Чем устранение состояния отличается от шага рекуррентности алгоритма Клини?
6. Вычислите производные ab^*a по буквам a b a: принимается ли слово?
7. Почему дополнение строят сменой принимающих состояний ДКА, а не НКА?

Что почитать

- m23 — «Регулярные выражения», «Свойства замкнутости», «Лемма о накачке».
- m23 — «Регулярные выражения на практике: ДКА против backtracking».